

Capitulo 4. Core APIs

1. F. Does not compile porque pretende guardar un int en un String.

```
int numFish= 4;
```

```
String anotherFish = numFish + 1;
```

Java evalúa primero el lado derecho, donde el + actúa como operador aritmético, ya que numFish y 1 son números.

2. C, E y F.

C porque usa el el nombre de la variable como si fuera el tipo.

```
(String beans [] = new beans [6]
```

E y F no dicen cuál va a ser el tamaño del array. En multidimensionales el primero a fuerza debe especificar tamaño, aunque los siguientes estén vacíos.

```
int [][] types = new int [];
```

```
int [][] java = new int [][];
```

3. A, C y D. Las tres opciones son fechas válidas, a diferencia de la B que dice 40 de marzo, la E que dice que hay 29 de febrero pero no es bisiesto, y la F tiene mal el nombre: debería ser `LocalDate.of(2022, Month.MARCH, 13)`; y no `MonthEnum.MARCH`.

4. A, C y D. A porque s y t sí son equals, porque comparten el mismo valor.

C porque usa `t.intern` por lo que regresa el valor del string pool.

El método `.intern()` le dice a Java: "Busca este valor en el String Pool y dame su referencia". Como "Hello" ya estaba en el pool (lo puso s), `t.intern()` devuelve exactamente esa misma dirección de memoria. Ahora ambos apuntan al mismo objeto del pool.

D porque porque también está comparando el contenido de algo que está en el string pool.

5. B porque `inster` usa dos parámetros: posición y valor, en qué posición se inserta el valor, pero no lo sustituye, lo recorre. Eso se llama `method chaining`, encadenar las llamadas. Entonces

```
sb.append("aaa") - aa
```

a

`.insert(1, "bb")` - abbaa

`.insert(4, "ccc")` - abbacca

6. C, dos líneas no van a compilar

24: `int two = Math.round(1.0);` // porque el método round devuelve int cuando usas float, no double

25: `float three = Math.random();` // El método random devuelve double, no float.

7. A y E. La primera es antes y están a seis horas de diferencia. Es más fácil si se convierte a GMT (Greenwich Mean Time), restando la TimeZone.

`2022-08-28T05:00 GMT-04:00` // $5 - (-4) = 9$

`2022-08-28T09:00 GMT-06:00` // $9 - (-6) = 15$

$15 - 9 = 6$ // 6 horas de diferencia.

8. A, B y F.

A. `builder.charAt(4)` // el index 4 es el 5 "12345"

B. `builder.replace(2, 4, "6").charAt(3)` // aquí primero se reemplaza el index 2 y 3, antes del 4, por el 6 "1265", y ahora el caracter en el index 3 es el 5

F. `string.replace("123", "1").charAt(2)` // primero se reemplaza el 123 por el 1, y queda 145, y otra vez nos pide el index 2, que es 5.

9. A, C y F. El primer elemento de un array está en el index 0, los arrays tienen un tamaño definido y cuando llamas `equals()` en dos diferentes arrays que contienen los mismos valores primitivos, siempre regresa false.

10. A, no hay errores de compilación. `min()` y `floor()` regresan el mismo tipo que se pasa. `round()` regresa long cuando se le pasa un double.

11. E. no compila porque está intentando usar una medida de tiempo (hours) en una sentencia de fecha (LocalDate)

12. A, D y E. Primero `indent` agrega un espacio al principio de la línea, después `stripLeading` borra los espacios del principio, entonces se cancelan. Después imprime la substring del index 1 hasta antes del 3 (12), la segunda dice que substring del 7 al 7, por lo que es una línea en blanco. Y la última dice que de la substring a partir del 7 y hacia el final, entonces es 78.

13. B. Los !!! solo se le agregan al `StringBuilder`. `roar1` es String, entonces es INMUTABLE, al concatenarle el !!! en realidad hace otro objeto. Así que al imprimir `System.out.println(roar1 + " " + roar2);` solo se cambia `roar2`. (con el `append()`)

14. A y F. La Opción A (`Instant.now()`): Es correcta porque utiliza un método de fábrica estático para capturar el momento exacto actual en la zona horaria UTC (GMT). La Opción F (`zonedDateTime.toInstant()`): Es correcta porque `ZonedDateTime` posee toda la información necesaria (fecha, hora y zona horaria) para localizar un punto único en la línea del tiempo universal

Para obtener un `Instant` (un punto único en el tiempo universal), necesitas ahora mismo (`now()`) o un objeto que ya sepa su zona horaria (`ZonedDateTime`). Las fechas y horas "locales" no sirven porque son ambiguas a nivel global.

15. C y E. C porque con `.sort(arr)` acomodamos los elementos según su orden natural (Unicode): 1. Números, 2. MAYUSCULAS y 3. minúsculas, por lo que la opción `[123, PIG, pig]` es correcta. La opción E es correcta porque Java sigue una fórmula para obtener un valor de retorno cuando busca un valor que no existe en un array:

Formula para Valor de retorno = $-(\text{punto_de_inserción}) - 1$

Si seguimos el orden natural (`[123, PIG, pig]`) "Pippa" se insertaría en el index 2, entre PIG y pig. `[123, PIG, __, pig]`. Si seguimos la fórmula: $-(2) - 1 = -3$

16. A, B y G. Estado inicial = `"ewe\nsheep\t"`; normal son 11, con 2 caracteres de escape: `\n` y `\t`

`indent()` agrega 2 caracteres al principio de cada línea. En el ejemplo tenemos 2 líneas.

- **Regla de espacios:** El método identifica 2 líneas (`"ewe"` y `"sheep\t"`) y añade 2 espacios al inicio de cada una ($2 + 2 = 4$ espacios).
- **Regla de normalización:** Siempre añade un salto de línea (`\n`) al final de la cadena si no lo tiene. Como `base` termina en `t`, se añade este carácter `\n` extra (+1 carácter) Entonces son +5 caracteres (16)

El método `translateEscape()` busca secuencias de texto que parecen escapes y las convierte en carácter de control real. Entonces junta el `\` y la `t` y lo deja en `\t`, lo que resta $11 - 1 = 10$

1. En el estado inicial, `\t` son dos caracteres físicos (`\` y `t`). (`"ewe"` y `"sheep\t"`)
2. `indent(2)` suma 5 caracteres (4 espacios por las dos líneas + 1 salto de línea por normalización).
3. `translateEscapes()` resta 1 carácter al fusionar los dos caracteres de texto `\` y `t` en un solo tabulador real.

17. A y G. Cuando `substring()` tiene dos parámetros el primero indica el index en el que empieza la substring y el segundo indica "hasta antes de", por lo que en la opción A `letters.substring(1, 2)` significa a partir del index 1 y antes del 2, así que es un solo carácter: `"b"` `var letters = new StringBuilder("abcdefg");` Si las opciones tienen el

mismo número (6,6) o (2,2) son legales pero devuelven una string vacía. (6,5) lanza excepción porque no puedes especificar los index en orden reverso.

18. C y F. s1 crea una String de 4 caracteres porque las comillas de cierre del bloque de texto están en la misma línea que la palabra, así que no se añade un salto de línea. Después se llaman unos métodos sobre s1, pero como las Strings son inmutables, no afectan el contenido de s1. después cuando llegamos a s1= "two" sucede una concatenación de nuestro valor original, quedando en "purrtwo", que son 7 caracteres, y cuando imprime la longitud esa es la respuesta: 7 (C). Con s2 comenzamos con una cadena vacía y se le suma un int 2, un char 'c' y un boolean false, dando como resultado "2cfalse". En el primer if s2 == "2cfalse" devuelve falso porque la primera está en el Pool String y el segundo es un nuevo objeto. Por lo mismo, devuelve true cuando se hace equals().

19. A, B y D. Al comparar Arrays se devuelve un int positivo cuando los arrays son diferentes y el primer array es lexicográficamente más grande que el segundo. Entonces la opción A es correcta, pues en el primer array s1 es mayor que s2 (L va antes que P, por lo que P es mayor)

El método mismatch() devuelve un entero positivo cuando los arrays son diferentes a partir del index 1 o mayor. Las opciones que cumplen con esta regla son la B y D. En s4 encontramos un Null, la regla es que null es menor que cualquier String. Además el array mas corto es menor.

20. A y D. La opción A es correcta porque var dateTime2 = dateTime1.plus(1, ChronoUnit.HOURS); añade una hora a la hora inicial. La opción D es correcta porque da la nueva hora local (De la 1 se pasa a las 3 automáticamente).

21. A y C. A porque usa el método reverse() que cambia el orden del StringBuilder (que sí es mutable). La C porque primero agrega "vaj\$" con append() quedando como "JavavaJ\$" y después borra los index 0,1 y 2, quedando como "avaJ\$", luego con .deleteCharAt(puzzle.length() - 1) borra el index 4 (5 de la longitud menos 1, queda 4). y queda "avaJ"

22. A. Porque la fecha que marca el ejercicio es 2022 APRIL 3, y las fechas son **inmutables**, así que aunque diga que le suma días y años, el código lo ignora.

Capítulo 5. Methods

1. A y E. La opción A es correcta porque las variables de static y de instancia pueden ser marcadas como final. Y la opción E dice que los primitivos marcados final ya no se pueden modificar.
2. B y C. Porque son modificadores de acceso validos (Public no es, debe ser con minúscula). default solo se utiliza en métodos dentro de una interfaz.
3. A y D. Porque las otras opciones incluyen doble tipo de retorno y ponen primero el void, antes que los modificadores de acceso y especificadores opcionales.
4. A, B, C y E. Porque el 6 puede ser promovido a de int a long o double, así como puede hacerse autobox en Integer.
5. A, C y D. A y C son correctas porque los métodos void pueden tener un return pero solo si no intentan regresar algún valor. D es correcta porque devuelve el mismotipo de valor que declara: int.
6. A, B y F. porque el parametro de argumentos variables está declarado al final. La F porque no usa ningún tipo de argumento variable.
7. D y F. D porque pasa el parametro inicial (primer true) mas dos "true, true" que se meten al array de argumentos variables, quedando de tamaño 2. La F porque Pasa los parámetros iniciales y luego añade un array de tamaño 2. `juggle(true, new boolean[2]);`
8. D. Puedes usar los modificadores de acceso para permitir el acceso a todos los métodos, pero a ninguna variable de instancia.
9. B, C, D y F. Como las dos clases están en paquetes diferentes (uno en my.school y otro en my.city) el acceso privado y el acceso de paquete no compilan.
10. B. Las variables estáticas se inicializan en el orden de aparición, luego corre el bloque static {}, que puede sobrescribir el valor inicial. LENGTH primero es 5, pero después el bloque static lo cambia a 10.
11. B y E. Un método static no puede llamar a un método de instancia directamente, por lo que no compila, aunque exista instancia. Llamar un método static desde una referencia null no lanza NullPointerException, sino que Java ignora el objeto y usa el tipo de la variable para resolver la llamada. Con métodos estáticos, el objeto no importa, ni siquiera necesita existir. Solo importa el tipo declarado de la variable.
12. B. Cuando pones final a una variable, le dices a Java que esa variable no se puede mover. Con Effective final, tenemos una variable que no dice final explicitamente, pero se comporta como si tuviera la palabra final, porque nunca se reasigna después de su primera asignación, por eso hay dos variables que califican como effective final en el código: name y giraffe. Las lambdas y clases internas solo pueden usar variables que sean final o effective final.
13. D. Porque el inicializador de RopeSwing es de instancia (no estático), por lo que solo corre cuando se hace new, ni nunca se construye el objeto, no corre. Después length es estática, por lo que todos los objetos comparten el mismo valor y cada construcción lo actualiza. Como el inicializador de instancia nunca corre, el valor final viene solo de las contrucciones que sí ocurrieron, o sea rope2, que imprime 8.
14. E. Si una variable es static final debe asignarse exactamente una vez, ya sea en la linea de declaración o en el bloque static {}, nunca en los dos lugares, nunca después, y nunca en un método. Entonces 4 líneas no compilan: bench porque nunca se asigna en ninguno de los lugares válidos., name se asigna en la

declaración en en el bloque estático, por lo que ya serían dos veces. `rightRope` también se asigna en dos bloques `static {}` distintos.

15. B. Hay dos maneras válidas de hacer `static import`: `import static java.util.Collections.*;` (B) y `import static java.util.Collections.sort;` (que no está entre las opciones). La clases `Collections` en sí requiere un `import` normal, no estático, el `static import` solo aplica a miembros estáticos (métodos y variables) , no a clases. Los parámetros del método no tienen nada que hacer en un `import`. Además siempre va primero `import static`, y no al revés (`static import`).
16. E. Java busca el método más específico disponible al hacer `overloading`. Un `short` se promueve a `int`, entonces llama al `prin(int)`. Un `boolean` no tiene promoción numérica, pero se puede autoboxear en `Boolean`, que es un `Object`, entonces llama a `print(Object)`. Un `double` también se autoboxea a `Double`, que también es `Object`. El resultado es `int-Object-Object`.
17. Java nunca puede modificar una variable primitiva del caller desde adentro de un método. Java es `pass-by-value`, o sea que siempre pasa una copia del valor, no la variable original. Entonces aunque dentro del método `square` se modifique `x`, y se calcule `y=81`, la variable `value` de afuera nunca se toca y sigue valiendo 9.
18. B, D y E. Java es `pass-by-value`, pero con objetos lo que pasa es una copia de la referencia, no el objeto en sí. Esto significa que si dentro del método se reasigna la variable a un nuevo objeto, el caller no se entera, solo cambias la copia local de la referencia. Pero si llamas a un método sobre el objeto, sí afectas al caller, porque ambos apuntan al mismo objeto en memoria. O sea que no puedes cambiar a que apunta la variable del caller, pero sí puedes modificar el contenido del objeto al que apunta.
19. B, C y E. Las reglas de dónde se puede asignar cada tipo de variable son: una variable final de instancia solo se puede asignar exactamente una vez: en la declaración, en un inicializador de instancia, o en el constructor; si ya se asignó en la declaración, no se puede volver a tocar en ningún otro lugar. Una variable `static` puede ser accedida tanto desde inicializadores estáticos como de instancia. Una variable de instancia normal solo puede ser accedida desde inicializadores de instancia o constructores, un bloque `static {}` no tiene acceso a variables de instancia por que corre antes de que exista cualquier objeto. La regla clave es: lo estático no puede tocar lo de instancia, pero lo de instancia sí puede tocar lo estático.
20. A y E. Java sigue un orden de preferencia estricto al resolver `overloading`:
tipo exacto → promoción numérica → autoboxing → `varargs`
Un 100 es `int` y llama directo al método `int` (opcion A). Si ese método no existe, Java prefiere autoboxing a `varargs`, entonces elige el constructor `Integer`. Un 100L es `long`, que al no tener método específico sube `Object` (Opción E). Java nunca hace `narrowing` (conversión a un tipo más pequeño) automáticamente, pero sí sube por la jerarquía de herencia hasta encontrar un `match`.
21. B y D. Para que un método sea un `overloading` válido solo necesita cambiar la lista de tipos de parámetros, el nombre debe ser el mismo, el tipo de retorno y el modificador de acceso pueden ser cualquier cosa. La opción A no es `overloading` porque tiene los mismos tipos de parámetros que el original, o sea, misma firma. C y E no son `overload` porque cambian el nombre del método, eso es ya otro método. Las opciones F y G no compilan porque `varargs` solo puede haber uno por método, y debe ser siempre el último parámetro.